

Python Programming and Data Analysis in the Age of AI

A beginner-friendly teaching document for undergraduate students

Prepared for summer undergraduate instruction

2026-06-30

1 How to use this document

This document is designed for students who are new to programming. It uses Python as the main language for learning programming and then applies Python to data analysis and plotting.

The document can support a summer semester course, a short bootcamp, or a self-study module. It starts with the question “What is programming?” and gradually builds toward practical analysis with data tables, files, and **visualizations**.

Teaching suggestion

For a summer semester, this material can be taught in 8 to 12 weeks. Each major part can be a week or a two-class unit, depending on the pace of the course and how much practice time students need.

1.1 Learning goals

By the end of this document, students should be able to:

1. Explain what a program is and how Python runs code.
2. Use **variables**, expressions, functions, **conditionals**, loops, and collections.
3. Read error messages and debug small programs.
4. Work with files and structured data.
5. Use Python libraries such as `math`, `random`, `pathlib`, `csv`, `json`, `numpy`, `pandas`, and `matplotlib`.
6. Clean, summarize, filter, group, and visualize data.
7. Compare basic Python ideas with JavaScript and C#.
8. Use **AI** tools responsibly as programming assistants, not as replacements for understanding.

1.2 Suggested course structure

Unit	Topic	Main skills
1	What programming is	Algorithms, syntax, variables, expressions
2	Basic Python	Types, strings, input/output, operators
3	Control flow	<code>if</code> , <code>elif</code> , <code>else</code> , Boolean logic
4	Loops	<code>for</code> , <code>while</code> , <code>range</code> , <code>break</code> , <code>continue</code>
5	Collections	Lists, tuples, dictionaries, sets
6	Functions	Parameters, return values, scope, type hints
7	Debugging and files	Errors, exceptions, text files, CSV, JSON
8	Data analysis	NumPy, pandas, tabular thinking
9	Plotting	Matplotlib, chart types, visual clarity
10	Mini-projects	End-to-end analysis and presentation
11	Python vs. JS/C#	Similar concepts, different syntax and habits
12	AI-assisted programming	Prompting, verification, testing, ethics

2 Part I: Programming fundamentals

3 What is programming?

Programming is the practice of writing instructions that a computer can follow.

A program is not magic. It is a precise set of steps written in a programming language. The computer follows those steps exactly, even when the steps are wrong.

For example, a human instruction might be:

Calculate the average score for the class.

A computer needs something more precise:

```
scores = [82, 91, 77, 88, 95]
average = sum(scores) / len(scores)
print(average)
```

This program says:

1. Store five scores in a **list**.
2. Add the scores.
3. Count how many scores there are.
4. Divide the total by the count.
5. Print the result.

3.1 Programs, data, and algorithms

Three important ideas appear throughout the course.

Concept	Meaning	Example
Data	Information a program uses	Numbers, names, dates, tables
Algorithm	A step-by-step method	Sort values from smallest to largest
Program	An algorithm written in code	A Python file that reads data and makes a chart

An algorithm can exist before code. For example, the algorithm for finding the largest value in a **list** is:

1. Start with the first value as the largest value seen so far.
2. Look at each remaining value.
3. If the current value is larger than the largest value seen so far, replace it.
4. After checking all values, report the largest value.

In Python:

```
numbers = [12, 5, 19, 3, 14]

largest = numbers[0]

for number in numbers:
    if number > largest:
        largest = number

print(largest)
```

3.2 Syntax and semantics

Programming languages have syntax and semantics.

Syntax means the grammar of code. It answers: Is the code written in a form Python understands?

Semantics means the meaning of the code. It answers: Does the code do what we intended?

This code has a syntax error because the closing parenthesis is missing:

```
print("Hello"
```

This code has valid syntax but wrong semantics if the goal is to calculate an average:

```
scores = [80, 90, 100]
wrong_average = sum(scores) # forgot to divide by the number of scores
print(wrong_average)
```

A major part of learning programming is learning to diagnose both kinds of problems.

3.3 Why Python?

Python is commonly used for teaching because it has readable syntax and supports many styles of programming. It is also widely used in data analysis, automation, web development, scientific computing, machine learning, and artificial intelligence workflows.

Python is an interpreted language. In beginner terms, that means you can run Python code directly and see results quickly. Python also has a large ecosystem of packages, which are reusable tools written by other programmers.

3.4 A first Python program

```
print("Hello, Python!")
```

The word `print` is the name of a built-in function. A function is a reusable action. Here, the action displays text.

The text `"Hello, Python!"` is a string, which means a piece of text.

3.5 Comments

Comments are notes for humans. Python ignores comments when running the program.

```
# This is a comment.
# It explains what the code is doing.

name = "Maya"
print("Hello,", name)
```

Good comments explain why something is being done, not only what the code already says.

```
# Good: the curve is nonlinear, so a log scale makes the trend easier to see.

# Less useful: print the variable x.
print(x)
```

4 Running Python code

Students commonly run Python in three ways.

Method	Description	Good for
Interactive shell / REPL	Type one line at a time and immediately see results	Quick experiments
Script file	Save code in a <code>.py</code> file and run it	Reusable programs
Notebook / Quarto document	Mix code, text, output, and plots	Data analysis and teaching

This document is written in Quarto Markdown (`.qmd`). A Quarto document can contain formatted text and executable code cells. For example:

```
 ::: {.cell execution_count=8}
  ``` {.python .cell-code}
 print("This code cell can run in Quarto.")
  ```
  :::
```

In this teaching document, the global setting `execute: eval: false` means code cells are displayed but not automatically executed when rendered. In class, an instructor can change that setting or run code interactively.

4.1 Installing packages

Python includes many built-in tools, but data analysis usually needs external packages.

Common beginner packages:

| Package | Purpose |
|-------------------------|---|
| <code>numpy</code> | Fast numerical arrays and mathematical operations |
| <code>pandas</code> | Tables and data analysis |
| <code>matplotlib</code> | Plotting and visualization |

| Package | Purpose |
|---------|-----------------------|
| jupyter | Interactive notebooks |

A common command-line installation pattern is:

```
python -m pip install numpy pandas matplotlib jupyter
```

For a class, it is often easier to provide one controlled environment instead of asking every student to configure their computer from scratch. Options include a university JupyterHub, cloud notebooks, a prebuilt conda environment, or a shared `requirements.txt` file.

5 Variables and expressions

A variable is a name that refers to a value.

```
age = 20
name = "Alex"
gpa = 3.72
is_enrolled = True

print(name)
print(age)
```

A useful beginner mental model is:

A variable is a label attached to a value.

The assignment operator `=` stores a value under a name.

```
x = 10
x = x + 5
print(x)
```

The line `x = x + 5` means:

1. Look up the current value of `x`.
2. Add 5.
3. Store the new result back into `x`.

It does not mean mathematical equality. In mathematics, $x = x + 5$ is impossible. In Python, it is an instruction.

5.1 Expressions

An expression is a piece of code that produces a value.

```
2 + 3
10 * 4
"data" + " analysis"
len("Python")
```

A statement is a complete instruction.

```
message = "Welcome to Python"
print(message)
```

5.2 Naming variables

Good variable names make code easier to understand.

```
# Clear
student_count = 42
average_score = 88.5

# Unclear
sc = 42
x = 88.5
```

Python naming habits:

- Use lowercase words separated by underscores: `average_score`.
- Start with a letter or underscore, not a number.
- Avoid names that are already built in, such as `list`, `dict`, `sum`, or `type`.
- Choose names that describe the data, not just the data type.

6 Basic data types

A data type describes what kind of value something is and what operations make sense for it.

| Type | Meaning | Example |
|-----------------------|---------------------|--|
| <code>int</code> | Integer | <code>42</code> |
| <code>float</code> | Decimal number | <code>3.14</code> |
| <code>str</code> | Text string | <code>"hello"</code> |
| <code>bool</code> | Boolean truth value | <code>True</code> , <code>False</code> |
| <code>NoneType</code> | Absence of a value | <code>None</code> |

Use `type()` to inspect a value's type.

```
print(type(42))
print(type(3.14))
print(type("hello"))
print(type(True))
print(type(None))
```

6.1 Numbers

```
a = 10
b = 3

print(a + b)    # addition
print(a - b)    # subtraction
print(a * b)    # multiplication
print(a / b)    # division, always produces a float
print(a // b)   # integer division
print(a % b)    # remainder
print(a ** b)   # exponentiation
```

The % operator is useful for checking divisibility.

```
number = 17

if number % 2 == 0:
    print("even")
else:
    print("odd")
```

6.2 Floating-point numbers

Computers approximate many decimal values. This can surprise beginners.

```
print(0.1 + 0.2)
```

The result may not display exactly as 0.3. For most data analysis this is normal, but students should avoid testing floating-point values for exact equality when small rounding differences are possible.

```
value = 0.1 + 0.2
print(abs(value - 0.3) < 0.000001)
```

6.3 Strings

Strings are text values.


```
first_name = "Ada"
last_name = "Lovelace"
full_name = first_name + " " + last_name
print(full_name)
```

Strings can use single or double quotes.

```
course = 'Python Programming'
semester = "Summer"
```

Use whichever makes the string easier to read.

```
message = "It's a good day to learn Python."
```

6.4 String methods

A method is a function attached to an object. Strings have many useful methods.

```
text = "  Data Analysis with Python  "

print(text.lower())
print(text.upper())
print(text.strip())
print(text.replace("Python", "pandas"))
print(text.startswith("  Data"))
```

Strings are sequences, so you can index and slice them.

```
word = "Python"

print(word[0])    # first character
print(word[-1])   # last character
print(word[0:3])  # characters at index 0, 1, 2
print(word[3:])   # from index 3 to the end
```

Python uses zero-based indexing: the first position is index 0.

6.5 Formatted strings

Formatted strings, often called f-strings, make it easy to insert values into text.

```
name = "Sam"
score = 91.236
```

```
print(f"{name} scored {score} points.")
print(f"{name} scored {score:.1f} points.")
```

The format `:.1f` means display one digit after the decimal point.

6.6 Booleans

Booleans represent truth values.

```
is_raining = False
has_umbrella = True

print(is_raining)
print(has_umbrella)
```

Booleans are central to decision-making.

```
temperature = 32
is_freezing = temperature <= 32
print(is_freezing)
```

6.7 None

None represents the absence of a value.

```
middle_name = None
print(middle_name)
```

None is useful when a value is unknown, missing, not yet computed, or intentionally empty.

```
best_score = None

scores = [82, 91, 77]
for score in scores:
    if best_score is None or score > best_score:
        best_score = score

print(best_score)
```

7 Input and output

Output means information the program shows to the user. The most basic output tool is `print()`.

```
print("Welcome")
print(2 + 3)
```

Input means information the user gives to the program. The basic input tool is `input()`.

```
name = input("What is your name? ")
print(f"Hello, {name}!")
```

`input()` always returns a string. Convert it when you need a number.

```
age_text = input("How old are you? ")
age = int(age_text)
print(f"Next year you will be {age + 1}.")
```

A shorter version:

```
age = int(input("How old are you? "))
print(f"Next year you will be {age + 1}.")
```

Common conversion functions:

| Function | Converts to |
|----------------------|----------------|
| <code>int()</code> | Integer |
| <code>float()</code> | Decimal number |
| <code>str()</code> | String |
| <code>bool()</code> | Boolean |

Beginner warning

`bool("False")` is `True` because non-empty strings are treated as true. Do not use `bool()` to parse user text like "yes" or "no" without writing your own logic.

8 Practice: basic expressions

1. Create **variables** for your name, major, year in school, and favorite number. Print one sentence using all four variables.
2. Ask a user for a temperature in Celsius and convert it to Fahrenheit using $F = C * 9/5 + 32$.
3. Ask a user for three exam scores and print the average.
4. Write a program that asks for a number of minutes and converts it to hours and minutes.

Example solution for exercise 4:

```
total_minutes = int(input("Enter minutes: "))
hours = total_minutes // 60
```

```
minutes = total_minutes % 60
print(f"{total_minutes} minutes is {hours} hours and {minutes} minutes.")
```

9 Part II: Control flow

10 Making decisions with `if`

Programs often need to choose between different actions.

```
score = 87

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("Needs improvement")
```

The keywords mean:

| Keyword | Meaning |
|-------------------|---|
| <code>if</code> | Check the first condition |
| <code>elif</code> | Check another condition if previous conditions were false |
| <code>else</code> | Run this block if no previous condition was true |

10.1 Indentation

Python uses indentation to show which lines belong together.

```
age = 19

if age >= 18:
    print("You are an adult.")
    print("This line is also inside the if block.")

print("This line always runs.")
```

Indentation is not just style in Python. It changes the meaning of the program.

10.2 Comparison operators

| Operator | Meaning | Example |
|--------------------|--------------------------|------------------------|
| <code>==</code> | equal to | <code>x == 5</code> |
| <code>!=</code> | not equal to | <code>x != 5</code> |
| <code><</code> | less than | <code>x < 5</code> |
| <code><=</code> | less than or equal to | <code>x <= 5</code> |
| <code>></code> | greater than | <code>x > 5</code> |
| <code>>=</code> | greater than or equal to | <code>x >= 5</code> |

A common beginner mistake is confusing `=` and `==`.

```
x = 5      # assignment: store 5 in x
x == 5     # comparison: ask whether x equals 5
```

10.3 Boolean operators

Use `and`, `or`, and `not` to combine conditions.

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("Entry allowed")
else:
    print("Entry denied")
```

```
is_weekend = True
is_holiday = False

if is_weekend or is_holiday:
    print("No class today")
```

```
is_logged_in = False

if not is_logged_in:
    print("Please log in")
```

10.4 Truthiness

Python can treat values as true or false in conditions.

Usually false:

- False
- None
- 0
- 0.0
- "" empty string
- [] empty **list**
- {} empty dictionary

Usually true:

- Nonzero numbers
- Non-empty strings
- Non-empty lists
- Non-empty **dictionaries**

```
name = ""

if name:
    print(f"Hello, {name}")
else:
    print("Name is missing")
```

Use truthiness carefully. It is convenient, but explicit comparisons are sometimes clearer for beginners.

11 Loops

A loop repeats code.

Python has two main kinds of loops:

| Loop | Best when |
|-------------------------|--|
| <code>for</code> loop | You know what collection or range you want to iterate over |
| <code>while</code> loop | You want to repeat until a condition changes |

11.1 `for` loops

A `for` loop repeats once for each item in a sequence.

```
names = ["Ava", "Ben", "Chen"]

for name in names:
    print(f"Hello, {name}")
```

The variable `name` takes one value at a time from the **list**.

11.2 range()

`range()` creates a sequence of integers.

```
for i in range(5):  
    print(i)
```

This prints 0, 1, 2, 3, and 4.

Useful patterns:

```
for i in range(1, 6):  
    print(i)          # 1 through 5  
  
for i in range(2, 11, 2):  
    print(i)          # 2, 4, 6, 8, 10
```

11.3 Accumulating values

Loops often accumulate a result.

```
scores = [82, 91, 77, 88]  
  
total = 0  
for score in scores:  
    total = total + score  
  
average = total / len(scores)  
print(average)
```

Python also has built-in functions for common operations.

```
scores = [82, 91, 77, 88]  
print(sum(scores) / len(scores))
```

Students should learn both versions. The manual loop teaches the algorithm. The built-in version is usually the better practical code.

11.4 Counting with a loop

```
scores = [82, 91, 77, 88, 95, 69]  
  
passing_count = 0  
for score in scores:  
    if score >= 70:  
        passing_count = passing_count + 1
```

```
print(passing_count)
```

Shorter version:

```
scores = [82, 91, 77, 88, 95, 69]
passing_count = sum(1 for score in scores if score >= 70)
print(passing_count)
```

The shorter version is idiomatic, but the longer version is easier for beginners to understand first.

11.5 while loops

A `while` loop repeats as long as a condition is true.

```
count = 1

while count <= 5:
    print(count)
    count = count + 1
```

`while` loops are useful for repeated input, simulations, and situations where the number of repetitions is not known in advance.

```
password = ""

while password != "python123":
    password = input("Enter password: ")

print("Access granted")
```

Infinite loops

A `while` loop can run forever if the condition never becomes false. Make sure something inside the loop changes the condition.

11.6 break and continue

`break` exits a loop early.

```
numbers = [3, 7, 11, 18, 21]

for number in numbers:
    if number % 2 == 0:
```



```
print(f"First even number: {number}")
break
```

`continue` skips to the next iteration.

```
numbers = [3, -2, 7, -5, 10]

for number in numbers:
    if number < 0:
        continue
    print(number)
```

Use `break` and `continue` when they make code clearer. Avoid using them so heavily that the loop becomes hard to follow.

11.7 `enumerate()`

Use `enumerate()` when you need both an index and a value.

```
names = ["Ava", "Ben", "Chen"]

for index, name in enumerate(names):
    print(index, name)
```

Use `start=1` when displaying human-friendly numbering.

```
for rank, name in enumerate(names, start=1):
    print(f"{rank}. {name}")
```

11.8 `zip()`

Use `zip()` to loop over two or more sequences at the same time.

```
names = ["Ava", "Ben", "Chen"]
scores = [91, 85, 93]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

12 Practice: conditionals and loops

1. Write a program that asks for a score and prints the letter grade.
2. Write a program that prints every number from 1 to 100 divisible by 7.
3. Given a **list** of temperatures, count how many are above 90.
4. Ask the user to enter numbers until they type `done`, then print the average.

5. Write a guessing game where the computer has a secret number and the user guesses until correct.

Example solution for exercise 4:

```
total = 0
count = 0

while True:
    text = input("Enter a number or 'done': ")

    if text == "done":
        break

    number = float(text)
    total = total + number
    count = count + 1

if count > 0:
    print(f"Average: {total / count}")
else:
    print("No numbers were entered.")
```

13 Collections

Collections store multiple values.

| Collection | Ordered? | Mutable? | Typical use |
|--------------------|--------------------|----------|--------------------------------------|
| <code>list</code> | Yes | Yes | A sequence of values that may change |
| <code>tuple</code> | Yes | No | A fixed group of related values |
| <code>dict</code> | In insertion order | Yes | Key-value lookup |
| <code>set</code> | No | Yes | Unique values and membership tests |

14 Lists

A **list** stores an ordered sequence of values.

```
scores = [82, 91, 77, 88]
print(scores)
```

Lists can contain values of different types, but in data analysis lists usually contain values of the same kind.

```
items = ["notebook", 3, True]
```

14.1 Indexing and slicing lists

```
scores = [82, 91, 77, 88]

print(scores[0])    # first item
print(scores[-1])   # last item
print(scores[1:3])  # items at index 1 and 2
```

14.2 Changing lists

Lists are mutable, meaning they can be changed.

```
scores = [82, 91, 77]

scores.append(88)
print(scores)

scores[0] = 85
print(scores)

scores.remove(77)
print(scores)
```

Common **list** methods:

| Method | Description |
|----------------------------------|--|
| <code>.append(x)</code> | Add <code>x</code> to the end |
| <code>.extend(other_list)</code> | Add all items from another list |
| <code>.insert(i, x)</code> | Insert <code>x</code> at position <code>i</code> |
| <code>.remove(x)</code> | Remove the first matching value |
| <code>.pop()</code> | Remove and return the last item |
| <code>.sort()</code> | Sort the list in place |
| <code>.copy()</code> | Make a shallow copy |

14.3 Sorting lists

```
scores = [82, 91, 77, 88]

print(sorted(scores))    # returns a new sorted list
```

```
print(scores)          # original list is unchanged

scores.sort()          # changes the original list
print(scores)
```

Use `sorted()` when you want a new **list**. Use `.sort()` when you want to modify the existing list.

14.4 List comprehensions

A **list** comprehension creates a list from another sequence in a compact way.

```
numbers = [1, 2, 3, 4, 5]
squares = [number ** 2 for number in numbers]
print(squares)
```

With a condition:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = [number for number in numbers if number % 2 == 0]
print(even_numbers)
```

Equivalent loop:

```
even_numbers = []
for number in numbers:
    if number % 2 == 0:
        even_numbers.append(number)
```

Comprehensions are useful, but clarity is more important than cleverness.

15 Tuples

A tuple is an ordered collection that cannot be changed after creation.

```
point = (3, 4)
print(point[0])
print(point[1])
```

Tuples are often used for fixed groups of related values.

```
student_record = ("Ava", 91)
name, score = student_record
print(name)
print(score)
```

The assignment `name, score = student_record` is called unpacking.

```
x, y = (10, 20)
print(x)
print(y)
```

16 Dictionaries

A dictionary stores key-value pairs.

```
student = {
    "name": "Ava",
    "major": "Biology",
    "score": 91
}

print(student["name"])
print(student["score"])
```

A key is used to look up a value. **Dictionaries** are useful when each item has named attributes.

16.1 Adding and changing dictionary values

```
student = {
    "name": "Ava",
    "major": "Biology",
    "score": 91
}

student["year"] = "junior"
student["score"] = 93

print(student)
```

16.2 Safe lookup with `.get()`

```
student = {"name": "Ava", "score": 91}

print(student.get("score"))
print(student.get("email"))
print(student.get("email", "not provided"))
```

Using `student["email"]` would cause an error if the key does not exist. `.get()` returns `None` or a default value instead.

16.3 Looping through dictionaries

```
student = {
    "name": "Ava",
    "major": "Biology",
    "score": 91
}

for key in student:
    print(key, student[key])

for key, value in student.items():
    print(key, value)
```

16.4 Counting with dictionaries

Dictionaries are useful for frequency counts.

```
majors = ["Biology", "Math", "Biology", "History", "Math", "Biology"]

counts = {}
for major in majors:
    if major not in counts:
        counts[major] = 0
    counts[major] = counts[major] + 1

print(counts)
```

The standard library also provides `Counter` for this common pattern.

```
from collections import Counter

majors = ["Biology", "Math", "Biology", "History", "Math", "Biology"]
counts = Counter(majors)
print(counts)
```

17 Sets

A set stores unique values.

```
majors = {"Biology", "Math", "History", "Biology"}
print(majors)
```

The duplicate value appears only once.

Sets are useful for membership checks.

```

allowed_majors = {"Biology", "Math", "History"}
major = "Math"

if major in allowed_majors:
    print("Allowed")

```

They are also useful for comparing groups.

```

python_students = {"Ava", "Ben", "Chen", "Dana"}
stats_students = {"Chen", "Dana", "Eli"}

print(python_students & stats_students) # intersection
print(python_students | stats_students) # union
print(python_students - stats_students) # difference

```

18 Practice: collections

1. Create a **list** of five cities. Print the first and last city.
2. Given a **list** of numbers, create a new list containing only values greater than 10.
3. Create a dictionary for a book with title, author, year, and number of pages.
4. Given a **list** of words, count how many times each word appears.
5. Given two sets of students, find who is enrolled in both courses.

Example solution for exercise 4:

```

words = ["data", "python", "data", "ai", "python", "data"]

counts = {}
for word in words:
    counts[word] = counts.get(word, 0) + 1

print(counts)

```

19 Part III: Functions, errors, and files

20 Functions

A function is a reusable block of code that performs a task.

Functions help programmers:

- Avoid repeating code.
- Give names to important actions.
- Break large problems into smaller problems.
- Test pieces of a program separately.

20.1 Defining a function

```
def greet():  
    print("Hello!")  
  
greet()  
greet()
```

The keyword `def` begins a function definition. The function body is indented.

20.2 Parameters and arguments

A parameter is a variable listed in a function definition.

An argument is a value passed into the function when it is called.

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Ava")  
greet("Ben")
```

Here, `name` is the parameter. "Ava" and "Ben" are arguments.

20.3 Return values

Some functions print a result. Other functions return a result.

```
def square(number):  
    return number ** 2  
  
result = square(5)  
print(result)
```

`return` sends a value back to the code that called the function.

A beginner mistake is confusing `print()` and `return`.

```
def square_print(number):  
    print(number ** 2)  
  
def square_return(number):  
    return number ** 2
```



```
value1 = square_print(5)
value2 = square_return(5)

print("value1:", value1)
print("value2:", value2)
```

The first function displays a result but returns `None`. The second function returns a value that can be stored and reused.

20.4 Multiple parameters

```
def calculate_average(score1, score2, score3):
    return (score1 + score2 + score3) / 3

average = calculate_average(80, 90, 100)
print(average)
```

20.5 Default parameter values

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Ava"))
print(greet("Ava", "Good morning"))
```

20.6 Keyword arguments

```
def describe_student(name, major, year):
    return f"{name} is a {year} majoring in {major}."

print(describe_student(name="Ava", major="Biology", year="junior"))
```

Keyword arguments make function calls easier to read, especially when there are several arguments.

20.7 Scope

Scope describes where a variable can be used.

```
def calculate_tax(price):
    tax_rate = 0.08
    return price * tax_rate

print(calculate_tax(100))
# print(tax_rate) # This would cause an error because tax_rate is local to
the function.
```

A variable created inside a function is usually local to that function.

20.8 Type hints

Type hints describe the expected types of parameters and return values.

```
def calculate_average(scores: list[float]) -> float:
    return sum(scores) / len(scores)

print(calculate_average([80.0, 90.0, 100.0]))
```

Type hints do not usually change how Python runs the code. They help humans, editors, and checking tools understand the programmer's intention.

20.9 Docstrings

A docstring explains what a function does.

```
def celsius_to_fahrenheit(celsius: float) -> float:
    """Convert a temperature from Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32

print(celsius_to_fahrenheit(20))
```

Docstrings are useful for students because they force a clear statement of purpose.

20.10 Designing functions

A good beginner function usually has these qualities:

- It has one clear purpose.
- Its name starts with a verb when it performs an action: `calculate_average`, `load_data`, `plot_scores`.
- It uses parameters instead of relying on hidden global **variables**.
- It returns a value when the result should be reused.
- It is short enough to understand at a glance.

20.11 Example: from repeated code to a function

Repeated code:

```
score1 = 82
score2 = 91
score3 = 77
average1 = (score1 + score2 + score3) / 3
print(average1)

score4 = 88
score5 = 95
score6 = 84
average2 = (score4 + score5 + score6) / 3
print(average2)
```

Function version:

```
def average_three(a, b, c):
    return (a + b + c) / 3

print(average_three(82, 91, 77))
print(average_three(88, 95, 84))
```

More flexible version:

```
def average(scores):
    return sum(scores) / len(scores)

print(average([82, 91, 77]))
print(average([88, 95, 84, 90, 92]))
```

21 Errors and debugging

Debugging is the process of finding and fixing problems in code.

Students should learn that **errors** are normal. Experienced programmers read error messages constantly.

21.1 Common error types

| Error | Typical cause |
|--------------------------|------------------------------|
| <code>SyntaxError</code> | Python cannot parse the code |

| Error | Typical cause |
|--------------------------------|--|
| <code>NameError</code> | A variable or function name does not exist |
| <code>TypeError</code> | Operation used with an inappropriate type |
| <code>ValueError</code> | Type is correct, but value is invalid |
| <code>IndexError</code> | List index is out of range |
| <code>KeyError</code> | Dictionary key does not exist |
| <code>FileNotFoundError</code> | File path is wrong or file does not exist |
| <code>ZeroDivisionError</code> | Division by zero |

21.2 Reading a traceback

A traceback shows where Python encountered an error.

Example:

```
scores = [82, 91, 77]
print(scores[5])
```

This produces an `IndexError` because index 5 does not exist.

When reading a traceback:

1. Start at the bottom to identify the error type and message.
2. Look for the line number in your own code.
3. Read the line carefully.
4. Check the values of **variables** used on that line.

21.3 Debugging with `print()`

```
def average(scores):
    print("scores:", scores)
    print("count:", len(scores))
    return sum(scores) / len(scores)

print(average([82, 91, 77]))
```

Print debugging is simple and effective. It helps students see what the program is actually doing.

21.4 Debugging with small tests

```
def celsius_to_fahrenheit(celsius):
    return celsius * 9 / 5 + 32
```

```
assert celsius_to_fahrenheit(0) == 32
assert celsius_to_fahrenheit(100) == 212
```

`assert` checks that a condition is true. If not, Python raises an error.

Small tests are especially useful when using **AI**-generated code. The **AI** may produce code that looks plausible but has subtle mistakes.

21.5 Handling exceptions

Sometimes you expect an operation might fail, and you want the program to respond gracefully.

```
text = input("Enter a number: ")

try:
    number = float(text)
    print(f"Double: {number * 2}")
except ValueError:
    print("That was not a valid number.")
```

Do not use `try/except` to hide **errors** you do not understand. Use it when you know what might go wrong and how the program should respond.

22 Files and paths

Data analysis usually begins with data stored in files.

Python can read and write many kinds of files. Common beginner formats include:

| Format | Description | Common extension |
|------------|-----------------------|------------------|
| Plain text | Unstructured text | .txt |
| CSV | Comma-separated table | .csv |
| JSON | Nested key-value data | .json |

22.1 Paths with `pathlib`

A path describes where a file is located.

```
from pathlib import Path

project_folder = Path(".")
data_folder = project_folder / "data"
file_path = data_folder / "students.csv"
```

```
print(file_path)
```

Using `Path` is often clearer than manually joining strings.

22.2 Writing a text file

```
from pathlib import Path

path = Path("message.txt")
path.write_text("Hello from Python!\n")
```

22.3 Reading a text file

```
from pathlib import Path

path = Path("message.txt")
text = path.read_text()
print(text)
```

22.4 Reading line by line

```
from pathlib import Path

path = Path("message.txt")

for line in path.read_text().splitlines():
    print(line)
```

For small files, `read_text()` is convenient. For very large files, reading line by line with `open()` can be more memory-efficient.

```
with open("message.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

The `with` statement closes the file automatically when the block finishes.

22.5 CSV files

CSV means comma-separated values. It is a common format for tabular data.

Example CSV content:

```
name,major,score
Ava,Biology,91
Ben,Math,85
Chen,History,93
```

Python's standard library includes a `csv` module.

```
import csv

with open("students.csv", "r", encoding="utf-8", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], row["score"])
```

For larger data analysis tasks, `pandas` is usually more convenient.

```
import pandas as pd

students = pd.read_csv("students.csv")
print(students)
```

22.6 JSON files

JSON is useful for nested data.

```
import json

student = {
    "name": "Ava",
    "major": "Biology",
    "scores": [91, 88, 95]
}

with open("student.json", "w", encoding="utf-8") as file:
    json.dump(student, file, indent=2)
```

Reading JSON:

```
import json

with open("student.json", "r", encoding="utf-8") as file:
    student = json.load(file)
```

```
print(student["name"])
print(student["scores"])
```

23 Modules, packages, and imports

A module is a Python file that contains reusable code. A package is a collection of modules.

Use `import` to bring outside code into your program.

```
import math

print(math.sqrt(25))
print(math.pi)
```

Import a specific function:

```
from math import sqrt

print(sqrt(25))
```

Use an alias:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Aliases like `pd`, `np`, and `plt` are common in data analysis.

23.1 Useful standard library modules

| Module | Purpose |
|--------------------------|---|
| <code>math</code> | Mathematical functions |
| <code>random</code> | Random numbers and random choices |
| <code>statistics</code> | Basic statistics |
| <code>pathlib</code> | File paths |
| <code>csv</code> | CSV files |
| <code>json</code> | JSON files |
| <code>datetime</code> | Dates and times |
| <code>collections</code> | Specialized containers such as <code>Counter</code> |
| <code>itertools</code> | Efficient iteration tools |

23.2 Random numbers

```
import random

print(random.randint(1, 6))
print(random.choice(["heads", "tails"]))
```

Set a seed when you want reproducible random results.

```
import random

random.seed(42)
print(random.randint(1, 6))
```

Reproducibility is important in data analysis because another person should be able to rerun your analysis and get the same result when the process is deterministic.

24 Object-oriented programming: a gentle introduction

Python supports **object-oriented programming**. Beginners do not need to master it immediately, but they should understand what objects, attributes, and methods are.

An object is a value that can contain data and behavior.

```
text = "python"
print(text.upper())
```

Here, `text` is a string object, and `.upper()` is a method.

24.1 Classes

A class defines a new kind of object.

```
class Student:
    def __init__(self, name, major, score):
        self.name = name
        self.major = major
        self.score = score

    def passed(self):
        return self.score >= 70

student = Student("Ava", "Biology", 91)
print(student.name)
print(student.passed())
```

Key ideas:

| Term | Meaning |
|-------------------|--|
| Class | Blueprint for a kind of object |
| Object / instance | A specific value created from a class |
| Attribute | Data stored on an object |
| Method | Function attached to an object |
| <code>self</code> | The current object inside a class method |

For data analysis, students will use objects constantly, even if they rarely write their own classes. A **pandas** `DataFrame`, a matplotlib `Figure`, and a string are all objects.

25 Practice: functions, errors, and files

1. Write a function `is_even(number)` that returns `True` if a number is even.
2. Write a function `letter_grade(score)` that returns a letter grade.
3. Write a function `average(scores)` that handles an empty **list** by returning `None`.
4. Write a program that reads a text file and counts how many lines it contains.
5. Write a program that reads a CSV file of students and prints only students with scores above 90.

Example solution for exercise 3:

```
def average(scores):
    if len(scores) == 0:
        return None
    return sum(scores) / len(scores)

print(average([80, 90, 100]))
print(average([]))
```

26 Part IV: Data analysis with Python

27 Thinking like a data analyst

Data analysis is the process of using data to answer questions.

A typical workflow is:

1. Ask a question.
2. Collect or load data.
3. Inspect the data.
4. Clean the data.
5. Transform the data.

6. Summarize patterns.
7. Visualize results.
8. Communicate conclusions and limitations.

Programming matters because real **datasets** are often too large, messy, or repetitive to handle manually.

27.1 A small teaching dataset

The examples below use a small student **dataset**.

```
import pandas as pd

students = pd.DataFrame({
    "student_id": [1, 2, 3, 4, 5, 6, 7, 8],
    "name": ["Ava", "Ben", "Chen", "Dana", "Eli", "Fatima", "Gio", "Hana"],
    "major": ["Biology", "Math", "History", "Math", "Biology", "Computer Science",
              "History", "Computer Science"],
    "year": ["first", "second", "third", "second", "fourth", "third", "first",
             "fourth"],
    "hours_studied": [5.5, 3.0, 4.5, 6.0, 2.5, 7.0, 3.5, 6.5],
    "exam_score": [88, 76, 91, 85, 69, 95, 73, 90],
    "submitted_project": [True, True, True, False, False, True, True, True]
})

print(students)
```

28 NumPy basics

NumPy is a library for numerical computing. Its main data structure is the array.

A Python **list** is flexible. A **NumPy** array is designed for fast numerical operations.

```
import numpy as np

scores = np.array([88, 76, 91, 85, 69, 95, 73, 90])

print(scores)
print(scores.mean())
print(scores.max())
print(scores.min())
```

28.1 Vectorized operations

With a Python **list**, multiplying by 2 repeats the list.

```
values = [1, 2, 3]
print(values * 2)
```

With a **NumPy** array, multiplying by 2 multiplies each value.

```
import numpy as np

values = np.array([1, 2, 3])
print(values * 2)
```

This is called vectorization. It means applying an operation to many values at once without writing a manual loop.

28.2 Boolean filtering with NumPy

```
import numpy as np

scores = np.array([88, 76, 91, 85, 69, 95, 73, 90])
high_scores = scores[scores >= 90]

print(high_scores)
```

The expression `scores >= 90` produces a Boolean array.

```
print(scores >= 90)
```

NumPy is important because **pandas** uses similar ideas for columns and filters.

29 pandas basics

pandas is a library for working with tabular data.

The two most important **pandas** objects are:

| Object | Meaning |
|------------------------|-------------------------------|
| <code>Series</code> | One column of data |
| <code>DataFrame</code> | A table with rows and columns |

29.1 Creating a DataFrame

```
import pandas as pd

students = pd.DataFrame({
    "name": ["Ava", "Ben", "Chen"],
```

```

        "major": ["Biology", "Math", "History"],
        "score": [88, 76, 91]
    })

print(students)

```

29.2 Loading a CSV file

```

import pandas as pd

students = pd.read_csv("students.csv")

```

In a real project, data is often loaded from a CSV, Excel file, database, web API, or research data repository.

29.3 Inspecting a DataFrame

```

print(students.head())
print(students.tail())
print(students.shape)
print(students.columns)
print(students.info())
print(students.describe())

```

Common inspection methods:

| Method | Purpose |
|--------------------------|--|
| <code>.head()</code> | Show the first rows |
| <code>.tail()</code> | Show the last rows |
| <code>.shape</code> | Number of rows and columns |
| <code>.columns</code> | Column names |
| <code>.info()</code> | Column types and missing values |
| <code>.describe()</code> | Summary statistics for numeric columns |

29.4 Selecting columns

```

print(students["exam_score"])
print(students[["name", "exam_score"]])

```

One pair of brackets returns a `Series`.

Two pairs of brackets return a `DataFrame`.

```
score_series = students["exam_score"]
score_table = students[["exam_score"]]

print(type(score_series))
print(type(score_table))
```

29.5 Filtering rows

```
high_scores = students[students["exam_score"] >= 90]
print(high_scores)
```

Read this as:

Select rows from `students` where the `exam_score` column is at least 90.

Multiple conditions use `&` for and, `|` for or, and parentheses around each condition.

```
math_high_scores = students[
    (students["major"] == "Math") &
    (students["exam_score"] >= 80)
]

print(math_high_scores)
```

29.6 Sorting rows

```
print(students.sort_values("exam_score"))
print(students.sort_values("exam_score", ascending=False))
```

Sort by multiple columns:

```
print(students.sort_values(["major", "exam_score"], ascending=[True, False]))
```

29.7 Creating new columns

```
students["passed"] = students["exam_score"] >= 70
students["study_hours_per_point"] = students["hours_studied"] /
students["exam_score"]

print(students)
```

A new column can be based on existing columns.

```
students["score_category"] = "standard"
students.loc[students["exam_score"] >= 90, "score_category"] = "high"
students.loc[students["exam_score"] < 70, "score_category"] = "low"

print(students[["name", "exam_score", "score_category"]])
```

29.8 Grouping and summarizing

Grouping answers questions such as:

- What is the average exam score by major?
- How many students are in each year?
- What percentage submitted the project?

```
mean_score_by_major = students.groupby("major")["exam_score"].mean()
print(mean_score_by_major)
```

Count rows by group:

```
student_count_by_major = students.groupby("major").size()
print(student_count_by_major)
```

Multiple summaries:

```
summary = students.groupby("major").agg(
    average_score=("exam_score", "mean"),
    max_score=("exam_score", "max"),
    student_count=("student_id", "count"),
    average_hours=("hours_studied", "mean")
)

print(summary)
```

29.9 Missing data

Real data often has missing values.

```
import pandas as pd

survey = pd.DataFrame({
    "name": ["Ava", "Ben", "Chen", "Dana"],
    "age": [19, None, 21, 20],
    "hours_studied": [5.5, 3.0, None, 6.0]
})
```

```
print(survey)
print(survey.isna())
print(survey.isna().sum())
```

Common strategies:

| Strategy | Example | When it may make sense |
|----------------------|--------------------------|---|
| Leave missing values | Keep NaN | Analysis method can handle missing values |
| Drop rows | <code>.dropna()</code> | Missing rows are few and not systematic |
| Fill with a value | <code>.fillna(0)</code> | Missing means zero in context |
| Fill with summary | Fill with mean or median | Reasonable approximation for numeric data |
| Flag missingness | Add a Boolean column | Missingness itself may be informative |

Examples:

```
print(survey.dropna())

survey["hours_missing"] = survey["hours_studied"].isna()
survey["hours_studied_filled"]
= survey["hours_studied"].fillna(survey["hours_studied"].mean())

print(survey)
```

Data ethics note

Filling missing values can change conclusions. Always explain how missing values were handled.

29.10 Renaming columns

```
students = students.rename(columns={
    "exam_score": "score",
    "hours_studied": "study_hours"
})
```



```
print(students.columns)
```

29.11 Working with dates

```
import pandas as pd

attendance = pd.DataFrame({
    "student": ["Ava", "Ava", "Ben", "Ben"],
    "date": ["2026-06-01", "2026-06-03", "2026-06-01", "2026-06-03"],
    "present": [True, True, True, False]
})

attendance["date"] = pd.to_datetime(attendance["date"])
attendance["weekday"] = attendance["date"].dt.day_name()

print(attendance)
```

Date columns must often be converted before date operations work correctly.

29.12 Combining datasets

Data is often spread across multiple tables.

```
import pandas as pd

students = pd.DataFrame({
    "student_id": [1, 2, 3],
    "name": ["Ava", "Ben", "Chen"],
    "major": ["Biology", "Math", "History"]
})

scores = pd.DataFrame({
    "student_id": [1, 2, 3],
    "exam_score": [88, 76, 91]
})

combined = students.merge(scores, on="student_id", how="left")
print(combined)
```

Common join types:

| Join type | Meaning |
|-----------|-----------------------------------|
| left | Keep all rows from the left table |

| Join type | Meaning |
|--------------------|------------------------------------|
| <code>right</code> | Keep all rows from the right table |
| <code>inner</code> | Keep only matching rows |
| <code>outer</code> | Keep all rows from both tables |

29.13 Saving results

```
summary.to_csv("summary_by_major.csv")
```

You can also save a DataFrame without the index column.

```
summary.to_csv("summary_by_major.csv", index=False)
```

30 Example analysis: study time and exam scores

Question:

Do students who studied more hours tend to have higher exam scores?

First, create or load the data.

```
import pandas as pd

students = pd.DataFrame({
    "student_id": [1, 2, 3, 4, 5, 6, 7, 8],
    "name": ["Ava", "Ben", "Chen", "Dana", "Eli", "Fatima", "Gio", "Hana"],
    "major": ["Biology", "Math", "History", "Math", "Biology", "Computer Science",
              "History", "Computer Science"],
    "year": ["first", "second", "third", "second", "fourth", "third", "first",
             "fourth"],
    "hours_studied": [5.5, 3.0, 4.5, 6.0, 2.5, 7.0, 3.5, 6.5],
    "exam_score": [88, 76, 91, 85, 69, 95, 73, 90],
    "submitted_project": [True, True, True, False, False, True, True, True]
})
```

Inspect:

```
print(students.head())
print(students.describe())
```

Calculate correlation:

```
correlation = students["hours_studied"].corr(students["exam_score"])
print(f"Correlation: {correlation:.2f}")
```

Summarize by project submission:

```
summary = students.groupby("submitted_project").agg(
    average_score=("exam_score", "mean"),
    average_hours=("hours_studied", "mean"),
    count=("student_id", "count")
)

print(summary)
```

Interpretation:

- A positive correlation suggests that higher study time is associated with higher exam scores in this small **dataset**.
- This does not prove that studying caused higher scores.
- The **dataset** is tiny, so conclusions should be cautious.
- Other **variables**, such as prior preparation, course difficulty, work hours, and attendance, may matter.

31 Practice: pandas

Use the student **dataset** above.

1. Print only the `name` and `exam_score` columns.
2. Filter to students who studied at least 5 hours.
3. Find the average exam score by year.
4. Create a new column called `passed` where a score of 70 or higher is passing.
5. Count how many students passed and failed.
6. Sort students from highest to lowest score.
7. Save the filtered high-scoring students to a CSV file.

Example solution for exercises 2 through 5:

```
studied_5_hours = students[students["hours_studied"] >= 5]
print(studied_5_hours)

average_by_year = students.groupby("year")["exam_score"].mean()
print(average_by_year)

students["passed"] = students["exam_score"] >= 70
pass_counts = students["passed"].value_counts()
print(pass_counts)
```

32 Part V: Plotting and visualization

33 Why visualize data?

Tables are precise, but plots make patterns easier to see. A good **plot** helps answer a question.

Common reasons to make a **plot**:

- Compare groups.
- Show a trend over time.
- Show the distribution of a variable.
- Show the relationship between two **variables**.
- Detect unusual values.

Common chart choices:

| Question | Useful chart |
|--|---|
| How does one numeric variable vary? | Histogram |
| How do groups compare? | Bar chart or box plot |
| How does a value change over time? | Line chart |
| How are two numeric variables related? | Scatter plot |
| What are the parts of a whole? | Bar chart; sometimes pie chart for simple cases |

34 Matplotlib basics

Matplotlib is a widely used Python plotting library.

A common import is:

```
import matplotlib.pyplot as plt
```

34.1 Line plot

A line **plot** is useful for ordered data, especially time.

```
import matplotlib.pyplot as plt

weeks = [1, 2, 3, 4, 5]
average_scores = [78, 81, 83, 86, 88]

plt.plot(weeks, average_scores, marker="o")
plt.xlabel("Week")
plt.ylabel("Average score")
```

```
plt.title("Average score over time")
plt.show()
```

34.2 Scatter plot

A scatter **plot** shows the relationship between two numeric **variables**.

```
import pandas as pd
import matplotlib.pyplot as plt

students = pd.DataFrame({
    "hours_studied": [5.5, 3.0, 4.5, 6.0, 2.5, 7.0, 3.5, 6.5],
    "exam_score": [88, 76, 91, 85, 69, 95, 73, 90]
})

plt.scatter(students["hours_studied"], students["exam_score"])
plt.xlabel("Hours studied")
plt.ylabel("Exam score")
plt.title("Study time and exam score")
plt.show()
```

34.3 Bar chart

A **bar chart** compares categories.

```
import pandas as pd
import matplotlib.pyplot as plt

students = pd.DataFrame({
    "major": ["Biology", "Math", "History", "Math", "Biology", "Computer Science",
    "History", "Computer Science"],
    "exam_score": [88, 76, 91, 85, 69, 95, 73, 90]
})

average_by_major = students.groupby("major")["exam_score"].mean().sort_values()

plt.bar(average_by_major.index, average_by_major.values)
plt.xlabel("Major")
plt.ylabel("Average exam score")
plt.title("Average exam score by major")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

34.4 Histogram

A histogram shows the distribution of a numeric variable.

```
import matplotlib.pyplot as plt

scores = [88, 76, 91, 85, 69, 95, 73, 90, 82, 78, 87, 94]

plt.hist(scores, bins=5)
plt.xlabel("Exam score")
plt.ylabel("Number of students")
plt.title("Distribution of exam scores")
plt.show()
```

34.5 Box plot

A box **plot** summarizes a distribution and can show potential outliers.

```
import pandas as pd
import matplotlib.pyplot as plt

students = pd.DataFrame({
    "major": ["Biology", "Math", "History", "Math", "Biology", "Computer Science",
              "History", "Computer Science"],
    "exam_score": [88, 76, 91, 85, 69, 95, 73, 90]
})

students.boxplot(column="exam_score", by="major")
plt.suptitle("")
plt.title("Exam score by major")
plt.xlabel("Major")
plt.ylabel("Exam score")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

34.6 Plotting directly from pandas

pandas has convenient plotting methods built on matplotlib.

```
average_by_major.plot(kind="bar")
plt.xlabel("Major")
plt.ylabel("Average exam score")
plt.title("Average exam score by major")
plt.xticks(rotation=30, ha="right")
```

```
plt.tight_layout()
plt.show()
```

This is often faster for exploratory work. Matplotlib gives more control when polishing a figure.

35 Visual communication principles

A good **plot** should be understandable without a long explanation.

35.1 Use clear titles and labels

```
plt.xlabel("Hours studied")
plt.ylabel("Exam score")
plt.title("Students who studied more tended to score higher")
```

The title should describe the message, not just the chart type.

Weak title:

Scatter **plot**

Better title:

Study time and exam score show a positive association

35.2 Choose the right scale

Changing an axis scale can change how readers perceive a pattern. Avoid misleading axis choices.

For bar charts, starting the y-axis at zero is usually clearer because bar length visually represents quantity.

35.3 Avoid chart clutter

Too many labels, colors, legends, or grid lines can make a chart hard to read. A simple chart is often better than a decorative chart.

35.4 Separate exploration from presentation

Exploratory plots help you understand the data. Presentation plots help communicate a conclusion.

Exploratory **plot**:

```
students.plot()
plt.show()
```

Presentation **plot**:

```

average_by_major = students.groupby("major")["exam_score"].mean().sort_values()

plt.bar(average_by_major.index, average_by_major.values)
plt.xlabel("Major")
plt.ylabel("Average exam score")
plt.title("Average exam score differs by major in the sample")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()

```

36 Example visualization project

Question:

How do study hours, project submission, and exam scores relate in a small course **dataset**?

```

import pandas as pd
import matplotlib.pyplot as plt

students = pd.DataFrame({
    "student_id": [1, 2, 3, 4, 5, 6, 7, 8],
    "name": ["Ava", "Ben", "Chen", "Dana", "Eli", "Fatima", "Gio", "Hana"],
    "major": ["Biology", "Math", "History", "Math", "Biology", "Computer Science",
              "History", "Computer Science"],
    "year": ["first", "second", "third", "second", "fourth", "third", "first",
             "fourth"],
    "hours_studied": [5.5, 3.0, 4.5, 6.0, 2.5, 7.0, 3.5, 6.5],
    "exam_score": [88, 76, 91, 85, 69, 95, 73, 90],
    "submitted_project": [True, True, True, False, False, True, True, True]
})

plt.scatter(students["hours_studied"], students["exam_score"])
plt.xlabel("Hours studied")
plt.ylabel("Exam score")
plt.title("Study time and exam score")
plt.show()

```

Add group summaries:

```

summary = students.groupby("submitted_project")["exam_score"].mean()

plt.bar(["No project", "Submitted project"], summary.values)

```



```
plt.ylabel("Average exam score")
plt.title("Project submission and average exam score")
plt.show()
```

Write a short interpretation:

In this sample, students who submitted the project had a higher average exam score. Students who studied more hours also tended to have higher scores. Because the **dataset** is small and observational, this does not prove causation.

37 Practice: plotting

1. Make a histogram of exam scores.
2. Make a **bar chart** of the number of students in each major.
3. Make a scatter **plot** of hours studied and exam score.
4. Add a title, x-axis label, and y-axis label to each **plot**.
5. Write two sentences interpreting one plot.

Example solution for exercise 2:

```
major_counts = students["major"].value_counts()

plt.bar(major_counts.index, major_counts.values)
plt.xlabel("Major")
plt.ylabel("Number of students")
plt.title("Number of students by major")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

38 Part VI: Mini-projects

39 Mini-project 1: Gradebook analysis

39.1 Goal

Analyze a small class gradebook and answer:

1. What is the average final grade?
2. Which students are at risk?
3. Which assessment had the lowest average?
4. How should the instructor summarize performance visually?

39.2 Starter data

```
import pandas as pd

grades = pd.DataFrame({
    "student": ["Ava", "Ben", "Chen", "Dana", "Eli", "Fatima", "Gio", "Hana"],
    "homework": [92, 81, 88, 75, 70, 95, 78, 90],
    "midterm": [85, 76, 91, 72, 68, 94, 73, 89],
    "project": [93, 84, 90, 78, 65, 98, 80, 91],
    "final_exam": [88, 74, 92, 80, 67, 96, 76, 90]
})
```

39.3 Analysis tasks

```
grades["final_grade"] = (
    grades["homework"] * 0.25 +
    grades["midterm"] * 0.25 +
    grades["project"] * 0.25 +
    grades["final_exam"] * 0.25
)

print(grades)
print(grades["final_grade"].mean())
```

Find at-risk students:

```
at_risk = grades[grades["final_grade"] < 75]
print(at_risk[["student", "final_grade"]])
```

Find average score by assessment:

```
assessment_means = grades[["homework", "midterm", "project",
    "final_exam"]].mean()
print(assessment_means)
```

Plot:

```
assessment_means.plot(kind="bar")
plt.ylabel("Average score")
plt.title("Average score by assessment")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

39.4 Written interpretation template

The class average final grade was _____. The lowest average assessment was _____, suggesting that students may need additional support in _____. The at-risk students were _____. This analysis is limited because it only includes scores and does not include attendance, prior preparation, or other context.

40 Mini-project 2: Simple survey analysis

40.1 Goal

Analyze survey responses about study habits and technology use.

40.2 Starter data

```
import pandas as pd

survey = pd.DataFrame({
    "student_id": range(1, 11),
    "study_hours_per_week": [4, 6, 3, 8, 5, 10, 2, 7, 6, 9],
    "uses_ai_tool": [True, True, False, True, False, True, False, True, False, True],
    "confidence": [3, 4, 2, 5, 3, 5, 2, 4, 3, 5],
    "preferred_language": ["Python", "Python", "JavaScript", "Python", "C#", "Python", "JavaScript", "Python", "C#", "Python"]
})
```

40.3 Questions

1. What is the average confidence score?
2. Do students who use **AI** tools report different confidence levels?
3. What programming language is most preferred?
4. Is study time associated with confidence?

40.4 Example code

```
print(survey["confidence"].mean())
print(survey.groupby("uses_ai_tool")["confidence"].mean())
print(survey["preferred_language"].value_counts())
print(survey["study_hours_per_week"].corr(survey["confidence"]))
```

Plot:

```
language_counts = survey["preferred_language"].value_counts()
```

```
plt.bar(language_counts.index, language_counts.values)
plt.xlabel("Preferred language")
plt.ylabel("Number of students")
plt.title("Preferred programming language in the sample")
plt.show()
```

41 Mini-project 3: Personal data log

Students can collect a small **dataset** about themselves for one week. Example **variables**:

- Date
- Hours slept
- Study hours
- Exercise minutes
- Screen time
- Mood rating from 1 to 5

Example structure:

```
import pandas as pd

log = pd.DataFrame({
    "date": ["2026-06-01", "2026-06-02", "2026-06-03", "2026-06-04",
"2026-06-05"],
    "sleep_hours": [7.5, 6.0, 8.0, 7.0, 6.5],
    "study_hours": [2.0, 3.5, 1.5, 4.0, 3.0],
    "exercise_minutes": [30, 0, 45, 20, 10],
    "mood": [4, 3, 5, 4, 3]
})

log["date"] = pd.to_datetime(log["date"])
print(log)
```

Possible analysis:

```
print(log.describe())
print(log[["sleep_hours", "study_hours", "exercise_minutes", "mood"]].corr())
```

Possible **plot**:

```
plt.plot(log["date"], log["sleep_hours"], marker="o")
plt.xlabel("Date")
plt.ylabel("Sleep hours")
plt.title("Sleep hours over time")
```

```
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

Privacy note

Personal data projects should be optional or designed with synthetic data. Students should not be required to reveal sensitive personal information.

42 Mini-project 4: Reproducible analysis report

Students create a short Quarto report that includes:

1. A research question.
2. A short description of the **dataset**.
3. Data loading code.
4. At least three summary **statistics**.
5. At least two **visualizations**.
6. A short interpretation.
7. A limitations section.

Suggested report outline:

```
# Title
```

```
## Question
```

```
What question does this analysis answer?
```

```
## Data
```

```
Where did the data come from? What does each row represent?
```

```
## Methods
```

```
What cleaning and summary steps were used?
```

```
## Results
```

```
Include tables and plots.
```

```
## Interpretation
```

```
What did you find? What do the results not prove?
```

```
## Limitations
```

```
What are the weaknesses of the data or method?
```

43 Part VII: Comparing Python, JavaScript, and C

Python, JavaScript, and C# can all teach the same programming concepts: **variables**, functions, **conditionals**, loops, collections, and objects. The differences are mostly syntax, runtime environment, typing style, and common use cases.

43.1 Common uses

| Language | Common beginner context | Common professional uses |
|------------|--|--|
| Python | Data analysis, scripting, automation, scientific computing | Data science, AI, automation, backend services |
| JavaScript | Web pages and interactive browser code | Frontend web, backend with Node.js, full-stack apps |
| C# | Structured programming and software engineering | Windows apps, enterprise systems, games with Unity, backend services |

43.2 Variables

Python:

```
age = 20
name = "Ava"
```

JavaScript:

```
let age = 20;
const name = "Ava";
```

C#:

```
int age = 20;
string name = "Ava";
```

Key comparison:

| Feature | Python | JavaScript | C# |
|------------------------------|---------------------------|--|--|
| Semicolons | Not normally used | Usually optional but common | Required |
| Type declarations | Usually not required | Usually not required | Usually required, though <code>var</code> can infer type |
| Variable declaration keyword | None for basic assignment | <code>let</code> , <code>const</code> , <code>var</code> | Type name or <code>var</code> |

43.3 Printing output

Python:

```
print("Hello")
```

JavaScript:

```
console.log("Hello");
```

C#:

```
Console.WriteLine("Hello");
```

43.4 Conditionals

Python:

```
score = 87

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("Other")
```

JavaScript:

```
let score = 87;

if (score >= 90) {
    console.log("A");
} else if (score >= 80) {
```

```

    console.log("B");
} else {
    console.log("Other");
}

```

C#:

```

int score = 87;

if (score >= 90)
{
    Console.WriteLine("A");
}
else if (score >= 80)
{
    Console.WriteLine("B");
}
else
{
    Console.WriteLine("Other");
}

```

Key comparison:

| Feature | Python | JavaScript | C# |
|------------------------------|-------------------|----------------------|----------------------|
| Blocks | Indentation | Curly braces | Curly braces |
| Else-if keyword | <code>elif</code> | <code>else if</code> | <code>else if</code> |
| Parentheses around condition | Not required | Required | Required |

43.5 Loops

Python `for` loop:

```

for number in [1, 2, 3]:
    print(number)

```

JavaScript `for...of` loop:

```

for (const number of [1, 2, 3]) {
    console.log(number);
}

```

C# `foreach` loop:


```
foreach (int number in new int[] {1, 2, 3})
{
    Console.WriteLine(number);
}
```

Python while loop:

```
count = 1

while count <= 3:
    print(count)
    count += 1
```

JavaScript while loop:

```
let count = 1;

while (count <= 3) {
    console.log(count);
    count += 1;
}
```

C# while loop:

```
int count = 1;

while (count <= 3)
{
    Console.WriteLine(count);
    count += 1;
}
```

43.6 Functions

Python:

```
def add(a, b):
    return a + b
```

JavaScript:

```
function add(a, b) {
    return a + b;
}
```

C#:

```
int Add(int a, int b)
{
    return a + b;
}
```

Key comparison:

| Feature | Python | JavaScript | C# |
|-----------------------|---------------------|---------------------------------------|------------------------------|
| Function keyword | <code>def</code> | <code>function</code> or arrow syntax | Return type plus method name |
| Return type declared? | Optional type hints | No, unless using TypeScript | Yes |
| Braces? | No | Yes | Yes |

43.7 Lists and arrays

Python **list**:

```
scores = [88, 76, 91]
scores.append(85)
print(scores[0])
```

JavaScript array:

```
let scores = [88, 76, 91];
scores.push(85);
console.log(scores[0]);
```

C# array and **list**:

```
int[] scoresArray = new int[] {88, 76, 91};
Console.WriteLine(scoresArray[0]);

List<int> scoresList = new List<int> {88, 76, 91};
scoresList.Add(85);
Console.WriteLine(scoresList[0]);
```

43.8 Dictionaries, objects, and maps

Python dictionary:

```
student = {"name": "Ava", "score": 91}
print(student["name"])
```

JavaScript object:

```
const student = {name: "Ava", score: 91};
console.log(student.name);
```

C# dictionary:

```
Dictionary<string, object> student = new Dictionary<string, object>();
student["name"] = "Ava";
student["score"] = 91;
Console.WriteLine(student["name"]);
```

In C#, a class is often preferred for structured records:

```
public class Student
{
    public string Name { get; set; }
    public int Score { get; set; }
}
```

43.9 Typing style

Python is dynamically typed. A variable can refer to values of different types at different times, although changing types carelessly makes code confusing.

```
x = 10
x = "ten"
```

C# is statically typed. The type of a variable is usually known at compile time.

```
int x = 10;
// x = "ten"; // This is not allowed.
```

JavaScript is dynamically typed.

```
let x = 10;
x = "ten";
```

Static typing can catch some mistakes earlier. Dynamic typing can make quick exploration flexible. For data analysis, Python's flexibility is convenient, but students should still write clear, predictable code.

43.10 Indentation and braces

Python uses indentation to define blocks.

```
if True:
    print("inside block")
print("outside block")
```

JavaScript and C# use braces.

```
if (true) {
    console.log("inside block");
}
console.log("outside block");
```

The Python style encourages visually clean code, but it also means indentation **errors** matter.

43.11 What transfers across languages?

The most important programming ideas transfer well:

- Breaking a problem into steps.
- Naming **variables** clearly.
- Using **conditionals** for decisions.
- Using loops for repetition.
- Using functions for reusable logic.
- Using data structures to organize information.
- Reading documentation.
- Debugging systematically.
- Testing assumptions.

Syntax changes from language to language. Problem-solving habits transfer.

44 Part VIII: Programming in the age of AI

AI tools can help students learn programming, but they can also hide misunderstandings. The course should teach students how to use AI as a tutor, reviewer, and assistant while still requiring them to reason about code.

45 What AI can help with

AI tools are useful for:

- Explaining unfamiliar code.
- Generating small examples.

- Suggesting debugging strategies.
- Translating code from one language to another.
- Creating practice problems.
- Drafting comments or documentation.
- Suggesting test cases.
- Refactoring code for readability.

Example prompt:

I am new to Python. Explain this code line by line and ask me two questions to check my understanding.

Example prompt:

Here is my function. It gives the wrong answer for an empty **list**. Help me find the bug, but do not give me the final answer immediately.

Example prompt:

Create five beginner practice problems about `for` loops using lists of numbers. Include expected outputs but not solutions.

46 What AI cannot replace

Students still need to understand:

- What the code is supposed to do.
- Whether the output makes sense.
- Whether the code is appropriate for the data.
- Whether assumptions are valid.
- Whether the result is ethical and reproducible.
- Whether a chart or statistic supports the claim being made.

AI-generated code can be wrong, outdated, inefficient, insecure, or mismatched to the assignment. A student who cannot read the code cannot responsibly use it.

47 A responsible AI workflow for beginners

Use this workflow when **AI** helps with programming:

1. State the goal in your own words.
2. Write or sketch a first attempt.
3. Ask **AI** for targeted help, not a complete replacement.
4. Run the code.
5. Read every line.
6. Test with small examples where you know the answer.

7. Explain the final code in your own words.
8. Cite or disclose **AI** assistance if required by the course policy.

47.1 Example: asking AI for debugging help

Weak prompt:

Fix my code.

Better prompt:

I am trying to calculate the average of a **list** of scores in Python. My code returns a `ZeroDivisionError` when the list is empty. Explain why this happens and give me two possible beginner-friendly ways to handle it.

47.2 Example: asking AI for concept explanation

Weak prompt:

Explain Python.

Better prompt:

Explain the difference between a Python **list** and dictionary for a student who has only learned **variables** and loops. Include one example of when each is useful.

47.3 Example: asking AI for verification

Prompt:

Review the following code for possible logic **errors**. Do not rewrite it unless necessary. First explain what the code currently does, then **list** possible problems, then suggest small tests.

This encourages the **AI** to act as a reviewer instead of silently replacing the student's work.

48 Testing AI-generated code

Suppose **AI** gives this function:

```
def average(scores):  
    return sum(scores) / len(scores)
```

A student should test normal and edge cases.

```
assert average([80, 90, 100]) == 90
assert average([10]) == 10
```

The empty-**list** case needs a design decision.

```
# This will cause ZeroDivisionError.
# average([])
```

Possible revised function:

```
def average(scores):
    if len(scores) == 0:
        return None
    return sum(scores) / len(scores)
```

Test again:

```
assert average([80, 90, 100]) == 90
assert average([10]) == 10
assert average([]) is None
```

49 Academic integrity and AI

A course should give students a clear **AI** policy. A useful policy distinguishes between support and substitution.

Possible allowed uses:

- Asking for explanations of concepts.
- Asking for help interpreting error messages.
- Asking for additional practice problems.
- Asking for feedback on code the student wrote.
- Asking for help writing comments after understanding the code.

Possible restricted uses:

- Submitting code the student cannot explain.
- Generating an entire assignment without disclosure.
- Using **AI** to bypass required practice.
- Fabricating data, sources, or results.

Example disclosure statement:

I used an **AI** assistant to help understand a `KeyError` and to suggest test cases. I wrote the final code and verified the output.

50 Part IX: Good programming habits

51 Write readable code

Readable code is easier to debug, share, and grade.

Less readable:

```
x=[82,91,77,88]
y=0
for z in x:y+=z
print(y/len(x))
```

More readable:

```
scores = [82, 91, 77, 88]

total = 0
for score in scores:
    total += score

average_score = total / len(scores)
print(average_score)
```

52 Prefer clarity over cleverness

A clever one-line solution can be impressive, but it may not be the best teaching code.

Compact:

```
print(sum(x for x in scores if x >= 70) / len([x for x in scores if x >= 70]))
```

Clearer:

```
passing_scores = [score for score in scores if score >= 70]
average_passing_score = sum(passing_scores) / len(passing_scores)
print(average_passing_score)
```

53 Use small steps

When solving a problem, do not write 50 lines before running anything. Build and test small pieces.

Example workflow:

1. Load the data.
2. Print the first few rows.

3. Select one column.
4. Create one filter.
5. Calculate one summary.
6. Make one **plot**.
7. Combine into a final report.

54 Make code reproducible

A reproducible analysis can be rerun by another person.

Good habits:

- Keep raw data separate from cleaned data.
- Use relative paths such as `data/students.csv` rather than computer-specific paths.
- Put imports at the top.
- Set random seeds when using randomness.
- Record package requirements for larger projects.
- Explain **data cleaning** decisions.
- Avoid manual spreadsheet edits that are not documented.

55 Use meaningful file organization

Example project folder:

```
python-course-project/  
├─ data/  
│   ├─ raw/  
│   └─ cleaned/  
├─ notebooks/  
├─ scripts/  
├─ figures/  
├─ reports/  
└─ README.md
```

A small class project does not need every folder, but students should learn that organization matters.

56 Part X: Assessment ideas

57 Low-stakes checks

Use frequent small activities:

- Predict the output of a short code snippet.
- Fix one error in a program.
- Fill in missing code.
- Explain a line of code in plain English.

- Write one test case.
- Modify a working example.

58 Concept questions

1. What is the difference between `=` and `==`?
2. What is the difference between `print()` and `return`?
3. Why does Python use indentation?
4. When would you use a `for` loop instead of a `while` loop?
5. What is the difference between a **list** and a dictionary?
6. What does a function parameter do?
7. Why should we inspect data before analyzing it?
8. Why does correlation not prove causation?
9. What should you check before trusting **AI**-generated code?
10. What makes a **plot** misleading?

59 Code tracing exercise

Ask students to predict the output before running the code.

```
numbers = [2, 5, 8, 11]
result = []

for number in numbers:
    if number % 2 == 0:
        result.append(number * 10)
    else:
        result.append(number + 1)

print(result)
```

Expected output:

```
[20, 6, 80, 12]
```

60 Debugging exercise

Find and fix the **errors**.

```
scores = [82, 91, 77, 88]

def average(scores)
    total = 0
    for score in score:
        total = total + scores
```

```
    return total / len(scores)

print(average(scores))
```

Possible corrected version:

```
scores = [82, 91, 77, 88]

def average(scores):
    total = 0
    for score in scores:
        total = total + score
    return total / len(scores)

print(average(scores))
```

61 Short programming assignments

61.1 Assignment 1: Calculator basics

Write a program that asks the user for two numbers and prints:

- Sum
- Difference
- Product
- Quotient
- Remainder

Include clear labels in the output.

61.2 Assignment 2: Grade classifier

Write a program that asks for a score from 0 to 100 and prints a letter grade.

Add error handling for invalid input.

61.3 Assignment 3: List summary

Given a **list** of numbers, write functions that return:

- Minimum
- Maximum
- Average
- Count above a threshold

Do not use built-in `min`, `max`, or `sum` for the first version. Then write a second version using built-ins.

61.4 Assignment 4: Word counter

Ask the user for a sentence. Count how many times each word appears.

Optional extension: ignore capitalization and punctuation.

61.5 Assignment 5: CSV analysis

Given a CSV file of students, write a program that:

- Loads the data.
- Prints the number of rows.
- Prints the average score.
- Filters students above a threshold.
- Saves the filtered data.

61.6 Assignment 6: Data visualization report

Use a small **dataset** to create a report with:

- One table of summary **statistics**.
- One histogram.
- One scatter **plot** or **bar chart**.
- A short explanation of each figure.

62 Final project option

Students complete an end-to-end analysis project.

Required sections:

1. Research question
2. Data description
3. **Data cleaning**
4. Summary **statistics**
5. **Visualizations**
6. Interpretation
7. Limitations
8. Reproducibility notes
9. **AI** assistance disclosure, if applicable

Possible project topics:

- Course survey data
- Public transportation ridership
- Weather records
- Campus event attendance
- Sports **statistics**
- Movie or music **dataset**
- Simulated business sales data
- Personal learning log using synthetic or anonymized data

63 Part XI: Reference sheets

64 Python syntax quick reference

| Task | Syntax example |
|-----------------|--------------------------------------|
| Assign variable | <code>x = 10</code> |
| Print | <code>print(x)</code> |
| Comment | <code># comment</code> |
| If statement | <code>if x > 0:</code> |
| Else-if | <code>elif x == 0:</code> |
| Else | <code>else:</code> |
| For loop | <code>for item in items:</code> |
| While loop | <code>while condition:</code> |
| Function | <code>def name(parameter):</code> |
| Return | <code>return value</code> |
| List | <code>[1, 2, 3]</code> |
| Dictionary | <code>{"name": "Ava"}</code> |
| Set | <code>{1, 2, 3}</code> |
| Tuple | <code>(1, 2)</code> |
| Import | <code>import math</code> |
| Read CSV | <code>pd.read_csv("file.csv")</code> |
| Plot | <code>plt.plot(x, y)</code> |

65 Common beginner mistakes

| Mistake | Example | Fix |
|----------------------------------|--|--------------------------------------|
| Assignment instead of comparison | <code>if x = 5:</code> | <code>if x == 5:</code> |
| Missing colon | <code>if x > 0</code> | <code>if x > 0:</code> |
| Wrong indentation | Inconsistent spaces | Use four spaces per level |
| String instead of number | <code>"5" + 1</code> | <code>int("5") + 1</code> |
| Index out of range | <code>items[10]</code> | Check <code>len(items)</code> |
| Dictionary key missing | <code>student["email"]</code> | Use <code>.get()</code> or check key |
| Empty list average | <code>sum(scores) / len(scores)</code> | Handle empty list first |

| Mistake | Example | Fix |
|--|--|--|
| Confusing print and return | Function prints but returns None | Use <code>return</code> for reusable result |
| Forgetting parentheses in pandas filters | <code>df[df["x"] > 1 & df["y"] < 5]</code> | <code>df[(df["x"] > 1) & (df["y"] < 5)]</code> |
| Trusting AI output without tests | Copy and submit | Run, inspect, test, explain |

66 pandas quick reference

| Task | Code |
|----------------|--|
| Import pandas | <code>import pandas as pd</code> |
| Read CSV | <code>df = pd.read_csv("file.csv")</code> |
| First rows | <code>df.head()</code> |
| Dimensions | <code>df.shape</code> |
| Column names | <code>df.columns</code> |
| Summary stats | <code>df.describe()</code> |
| Select column | <code>df["column"]</code> |
| Select columns | <code>df[["col1", "col2"]]</code> |
| Filter rows | <code>df[df["score"] >= 90]</code> |
| Sort | <code>df.sort_values("score")</code> |
| New column | <code>df["passed"] = df["score"] >= 70</code> |
| Group average | <code>df.groupby("major")["score"].mean()</code> |
| Missing counts | <code>df.isna().sum()</code> |
| Drop missing | <code>df.dropna()</code> |
| Fill missing | <code>df.fillna(0)</code> |
| Save CSV | <code>df.to_csv("out.csv", index=False)</code> |

67 matplotlib quick reference

| Task | Code |
|--------------|--|
| Import | <code>import matplotlib.pyplot as plt</code> |
| Line plot | <code>plt.plot(x, y)</code> |
| Scatter plot | <code>plt.scatter(x, y)</code> |

| Task | Code |
|-----------------|--|
| Bar chart | <code>plt.bar(categories, values)</code> |
| Histogram | <code>plt.hist(values)</code> |
| X label | <code>plt.xlabel("Label")</code> |
| Y label | <code>plt.ylabel("Label")</code> |
| Title | <code>plt.title("Title")</code> |
| Rotate x labels | <code>plt.xticks(rotation=30, ha="right")</code> |
| Improve spacing | <code>plt.tight_layout()</code> |
| Show plot | <code>plt.show()</code> |

68 Instructor notes

69 Pacing suggestions

A fast 8-week version:

| Week | Focus |
|------|--|
| 1 | Programming concepts, variables, types |
| 2 | Conditionals and loops |
| 3 | Collections and functions |
| 4 | Errors, debugging, files |
| 5 | pandas basics |
| 6 | Grouping, cleaning, merging |
| 7 | Plotting and interpretation |
| 8 | Final project and AI-assisted review |

A fuller 12-week version:

| Week | Focus |
|------|-------------------------------------|
| 1 | What programming is; running Python |
| 2 | Variables, types, strings |
| 3 | Conditionals |
| 4 | Loops |
| 5 | Collections |
| 6 | Functions |

| Week | Focus |
|------|--|
| 7 | Debugging, testing, files |
| 8 | NumPy and pandas introduction |
| 9 | pandas cleaning and grouping |
| 10 | Plotting |
| 11 | Mini-project workshops and language comparison |
| 12 | Final presentations and AI reflection |

70 Suggested classroom pattern

For each new concept:

1. Explain the concept in plain language.
2. Show a tiny code example.
3. Ask students to predict the output.
4. Run the code.
5. Modify one piece and rerun.
6. Give a short practice task.
7. Discuss one common error.
8. Connect the concept to data analysis.

71 Suggested grading categories

| Category | Possible weight |
|-------------------------------|-----------------|
| Weekly coding practice | 25% |
| Concept quizzes | 15% |
| Data analysis mini-projects | 25% |
| Final project | 25% |
| Participation and code review | 10% |

72 Code review checklist for students

Before submitting code, students should answer:

1. Does the program run from top to bottom?
2. Are variable names clear?
3. Are functions used for repeated logic?
4. Are edge cases handled?
5. Are plots labeled clearly?
6. Are **data cleaning** steps explained?
7. Can I explain every line of code?

8. Did I test **AI**-generated or **AI**-edited code?
9. Did I disclose **AI** assistance if required?

73 Further learning resources

Useful official or commonly used resources for students:

- Python documentation: <https://docs.python.org/3/>
- Python tutorial: <https://docs.python.org/3/tutorial/>
- **pandas** documentation: <https://pandas.pydata.org/docs/>
- **NumPy** documentation: <https://numpy.org/doc/>
- Matplotlib documentation: <https://matplotlib.org/stable/users/index.html>
- Quarto documentation: <https://quarto.org/docs/>

74 Closing message for students

Programming is a practical skill. It is learned by writing code, making mistakes, reading **errors**, testing ideas, and revising solutions.

Python is not only a language for professional programmers. It is also a tool for thinking clearly about data. In the age of **AI**, the most valuable skill is not memorizing every syntax detail. The valuable skill is knowing how to ask precise questions, evaluate answers, test code, interpret results, and communicate honestly.